



## REPORTE DE CIBERSEGURIDAD

# Controles de seguridad implementados en la plataforma

Resumen de las protecciones que ya están activas en la aplicación web de JTP Logistics, qué hace cada una y de qué amenaza nos protege.

|                   |                                                  |
|-------------------|--------------------------------------------------|
| Sistema           | Plataforma de gestión logística JTP (web)        |
| Alcance           | Controles de seguridad completados y verificados |
| Controles activos | 12 controles implementados                       |
| Fecha de emisión  | 3 de junio de 2026                               |
| Clasificación     | Confidencial — uso del cliente                   |

Este documento describe únicamente los controles de seguridad que ya están implementados, probados y activos en la aplicación. No incluye trabajo pendiente ni recomendaciones futuras. La tecnología base (Next.js, Better Auth, Prisma sobre PostgreSQL) sigue prácticas de seguridad estándar de la industria.

## Controles de seguridad implementados

A continuación se detalla cada protección activa: en qué consiste y un ejemplo concreto de la amenaza frente a la que protege a la operación y a los datos de JTP Logistics.

### 1 Autenticación con contraseñas cifradas

IMPLEMENTADO

#### QUÉ ES

El acceso al sistema se realiza con correo y contraseña gestionados por Better Auth. Las contraseñas nunca se guardan como texto; se almacenan con un algoritmo de derivación criptográfica (hashing con sal). Se exige una longitud mínima de 8 caracteres al registrarse o restablecer la clave.

#### EJEMPLO — DE QUÉ NOS PROTEGE

Si un atacante lograra ver la base de datos, no encontraría las contraseñas reales de los usuarios, solo valores cifrados imposibles de revertir. Protege contra el robo de credenciales en caso de fuga de datos.

### 2 Control de acceso por roles (RBAC)

IMPLEMENTADO

#### QUÉ ES

Cada usuario tiene un rol (administrador, transportista, colaborador, vendedor) y cada pantalla y operación del servidor verifica el rol antes de responder. Las funciones del servidor exigen explícitamente el rol correcto y devuelven 'No autorizado' (401) o 'Prohibido' (403) cuando no corresponde.

#### EJEMPLO — DE QUÉ NOS PROTEGE

Un transportista que intente abrir o llamar directamente a una pantalla de administrador —por ejemplo el listado de finanzas o de colaboradores— recibe un rechazo del servidor. Protege contra la escalada de privilegios y el acceso a datos de otras áreas.

### 3 Endpoints del servidor protegidos por defecto

IMPLEMENTADO

#### QUÉ ES

Las rutas de la API se envuelven en validadores de sesión y rol centralizados. Una petición sin sesión válida se rechaza antes de tocar la base de datos, devolviendo el código correcto (401/403) en lugar de un error genérico del servidor.

#### EJEMPLO — DE QUÉ NOS PROTEGE

Si alguien copia la URL de una API e intenta consumirla sin haber iniciado sesión, no obtiene datos ni un error que revele detalles internos: solo un 'No autorizado'. Protege contra el acceso no autenticado y la fuga de información por mensajes de error.

4

## Límite de intentos (rate limiting) en autenticación

IMPLEMENTADO

### QUÉ ES

Las rutas sensibles de acceso tienen un límite de peticiones por IP y por minuto: 10 intentos de inicio de sesión, y 5 de registro, recuperación y restablecimiento de contraseña. Al superarlo se responde 429 ('Demasiadas solicitudes').

#### EJEMPLO — DE QUÉ NOS PROTEGE

Un bot que intente adivinar una contraseña probando miles de combinaciones queda bloqueado tras unos pocos intentos por minuto. Protege contra ataques de fuerza bruta y de relleno de credenciales (credential stuffing).

5

## Protección contra CSRF y orígenes no confiables

IMPLEMENTADO

### QUÉ ES

La autenticación solo acepta peticiones provenientes de orígenes explícitamente confiables (el dominio de producción y los configurados). Las peticiones desde sitios desconocidos se rechazan.

#### EJEMPLO — DE QUÉ NOS PROTEGE

Si un sitio malicioso intenta enviar una petición en nombre de un usuario que ya tiene sesión abierta en JTP, el servidor la descarta por venir de un origen no confiable. Protege contra ataques de tipo Cross-Site Request Forgery (CSRF).

6

## Cookies de sesión seguras (HttpOnly)

IMPLEMENTADO

### QUÉ ES

La sesión se mantiene en cookies gestionadas por el servidor (vía el plugin oficial de Next.js de Better Auth), no accesibles desde el JavaScript de la página, con caché de sesión controlada del lado del servidor.

#### EJEMPLO — DE QUÉ NOS PROTEGE

Aunque se colara un script malicioso en el navegador, no podría leer ni robar la cookie de sesión para suplantar al usuario. Protege contra el secuestro de sesión (session hijacking) por XSS.

7

## Restablecimiento de contraseña seguro

IMPLEMENTADO

### QUÉ ES

El restablecimiento se hace por un enlace enviado al correo del usuario, con un token que expira en 1 hora. Al cambiar la contraseña, todas las sesiones activas se revocan automáticamente.

#### EJEMPLO — DE QUÉ NOS PROTEGE

Si alguien obtuvo acceso indebido a una cuenta, en cuanto el dueño legítimo restablece su contraseña, cualquier sesión del intruso queda invalidada de inmediato. Protege contra el acceso persistente tras un compromiso de cuenta.

8

## Prevención de inyección SQL

IMPLEMENTADO

### QUÉ ES

Todo el acceso a datos se hace mediante el ORM Prisma con consultas parametrizadas; no se construyen consultas SQL concatenando texto del usuario. La conexión a la base de datos exige cifrado TLS (sslmode=require).

#### EJEMPLO — DE QUÉ NOS PROTEGE

Si un usuario escribe código SQL dentro de un campo de búsqueda o formulario, se trata como texto literal y nunca se ejecuta como comando. Protege contra ataques de inyección SQL, una de las vulnerabilidades más explotadas a nivel mundial.

9

## Validación de entrada y anti-abuso en registro

IMPLEMENTADO

### QUÉ ES

Los formularios validan los datos tanto en el cliente como en el servidor (campos obligatorios, formato de correo, coincidencia de contraseñas). El registro aplica además heurísticas que rechazan nombres y correos con patrones de basura/automatizados, y los cuerpos JSON de la API se validan antes de procesarse.

#### EJEMPLO — DE QUÉ NOS PROTEGE

Un bot que intente crear cuentas masivas con nombres aleatorios tipo 'xkqzfg' o correos sospechosos es rechazado en el alta. Protege contra el registro automatizado abusivo y los datos malformados que podrían provocar fallos.

**10 Protección contra duplicados por condición de carrera**

IMPLEMENTADO

**QUÉ ES**

El número de cotización tiene una restricción de unicidad a nivel de base de datos (@unique), lo que impide que dos procesos simultáneos generen el mismo folio.

**EJEMPLO — DE QUÉ NOS PROTEGE**

Si dos usuarios generan una cotización en el mismo instante, la base de datos garantiza que no se asigne el mismo número a ambas. Protege contra la corrupción de datos y la duplicación de folios por accesos concurrentes (race conditions).

**11 Gestión segura de secretos y credenciales**

IMPLEMENTADO

**QUÉ ES**

Las claves sensibles (secreto de autenticación, cadena de conexión a la base de datos, credenciales de Cloudinary) viven en variables de entorno fuera del repositorio. Los archivos .env están excluidos del control de versiones y solo se publica una plantilla sin valores reales.

**EJEMPLO — DE QUÉ NOS PROTEGE**

Las contraseñas de servicios y la clave maestra de sesiones nunca quedan expuestas en el código que se sube a GitHub. Protege contra la fuga de credenciales por exposición accidental del repositorio.

**12 Integración continua con verificaciones automáticas**

IMPLEMENTADO

**QUÉ ES**

Cada cambio de código pasa por un pipeline de CI que ejecuta verificación de tipos (TypeScript), análisis estático (ESLint 9), pruebas automatizadas (Vitest sobre la lógica crítica) y compilación, antes de poder integrarse.

**EJEMPLO — DE QUÉ NOS PROTEGE**

Un cambio que introduzca un error de tipo o rompa una regla de cálculo de cotizaciones es detectado y bloqueado automáticamente antes de llegar a producción. Reduce la superficie de fallos y errores de seguridad introducidos por código defectuoso.

## Resumen ejecutivo

La plataforma cuenta actualmente con 12 controles de seguridad activos, que cubren acceso e identidad, autorización por roles, integridad de las sesiones, protección de la base de datos, validación de datos y calidad del código.

| #  | Control                                               | Área                           | Estado |
|----|-------------------------------------------------------|--------------------------------|--------|
| 1  | Autenticación con contraseñas cifradas                | Acceso / Identidad             | Activo |
| 2  | Control de acceso por roles (RBAC)                    | Autorización                   | Activo |
| 3  | Endpoints del servidor protegidos por defecto         | Autorización                   | Activo |
| 4  | Límite de intentos (rate limiting) en autenticación   | Defensa anti-abuso             | Activo |
| 5  | Protección contra CSRF y orígenes no confiables       | Integridad de sesión           | Activo |
| 6  | Cookies de sesión seguras (HttpOnly)                  | Integridad de sesión           | Activo |
| 7  | Restablecimiento de contraseña seguro                 | Acceso / Identidad             | Activo |
| 8  | Prevención de inyección SQL                           | Base de datos                  | Activo |
| 9  | Validación de entrada y anti-abuso en registro        | Validación de datos            | Activo |
| 10 | Protección contra duplicados por condición de carrera | Integridad de datos            | Activo |
| 11 | Gestión segura de secretos y credenciales             | Configuración                  | Activo |
| 12 | Integración continua con verificaciones automáticas   | Calidad / Cadena de suministro | Activo |

Conclusión: las protecciones fundamentales de una aplicación web de gestión —autenticación, control de acceso, defensa contra fuerza bruta, inyección SQL y CSRF, y manejo seguro de secretos— están implementadas y operativas. Esto establece una base de seguridad sólida sobre la cual seguir endureciendo el sistema conforme crezca la operación.